Project 8 Presentation Script

使用说明

- 课程安排：20 分钟 presentation，之后 5 分钟 Q&A。
- 下面的英文句子可以直接照读。
- [ ] 表示切到下一页。
- [ ] 表示只用中文解释这个术语或重点，不需要把整段英文翻译一遍。
- [DEMO] 表示现场操作。Demo 最好控制在 2–3 分钟。
- 如果现场 demo 超过 60 秒没有输出，直接使用文末的备用说法，不要现场调环境。
- 预估总时长约 18–19 分钟，留出停顿、切页和临时延迟。

上台前准备

1. 打开 `presentation/project8_spark_presentation.pptx`，从第 1 页开始放映。
2. 提前打开项目文件夹 `project8-spark`，把终端放在可以快速输入命令的位置。
3. 如果现场运行 demo，提前确认 `scripts/run_demo.ps1` 可以启动。不要在台上第一次安装 Java 或 PySpark。
4. 把以下结果文件作为备用证据打开，或者准备好截图：`results/summary.csv`、`results/correctness_cached.json`、`results/cache_speedup.png`。
5. Demo 不需要展示完整代码。只需要展示启动信息、缓存开关、运行时间和 correctness 结果。

Slide 1 — Spark caching is a tradeoff

时间：0:00–1:30

> [切到第 1 页]

Good morning everyone. My topic is Project 8, Spark caching and distributed execution.

The main question of this project is simple: when does caching actually help an iterative Spark workload?

Caching sounds like an automatic performance improvement, but it has a cost. Spark must first compute the data, materialize it, and keep partitions in memory. Because of that, caching may help later iterations, but the total runtime may still stay the same or even become slower for a small workload.

To study this tradeoff, I use PageRank as an iterative workload. I compare the same computation with caching disabled and with `MEMORY_ONLY` caching enabled.

My final conclusion is that caching helps when later iterations reuse enough data, but it is not automatically faster for every dataset size.

> [中文解释]

这里补一句中文即可：

“这次不是只证明 cache 会变快，而是要测量 cache 的收益和它第一次建立缓存的成本。”

> [切到第 2 页]

Slide 2 — Four observable requirements

时间：1:30–3:30

This project has four observable requirements.

First, the experiment must run one Spark driver and at least two worker processes or executors. I use a documented local Spark cluster with two workers.

Second, the workload must be iterative. PageRank is a good example because every iteration uses the result from the previous iteration.

Third, I need a fair comparison between caching enabled and caching disabled. The input data, number of partitions, number of iterations, and cluster configuration stay the same. Only the caching behavior changes.

Fourth, I need evidence from both measurements and Spark behavior. I collect runtime results, the number of partitions, the stages used by the job, and the storage level reported by Spark.

The purpose is to connect a performance number to a distributed-systems mechanism. In other words, I want to explain not only which run is faster, but also why.

> [中文解释]

“Fair comparison”可以解释为：“两组实验除了 cache 开关以外，其他条件都一样。”

“Executor”可以解释为：“真正执行 task、保存 partition 的 Spark worker 进程。”

> [切到第 3 页]

Slide 3 — Architecture

时间：3:30–5:30

This slide shows the execution architecture.

The input is a directed edge list. The Spark driver loads this input, creates the RDDs, and coordinates the PageRank iterations.

The local cluster has two worker processes. The partitions are distributed across these workers, and the workers execute the tasks for the PageRank computation.

The important object in this experiment is the graph structure, represented as an RDD. When caching is enabled, the graph RDD uses `MEMORY_ONLY` persistence. When caching is disabled, Spark recomputes the required data from its lineage whenever a later action needs it.

The driver records the runtime of the initial materialization separately from the runtimes of the later iterations. This separation is important because the first iteration includes the cost of building the cache.

> [中文解释]

“RDD”这里翻译成：“Spark 的分布式数据集合，可以被切成多个 partition，分布在不同 worker 上。”

“Lineage”这里翻译成：“Spark 保存的计算血缘，也就是这个 RDD 是怎样从输入一步步计算出来的。”

> [切到第 4 页]

Slide 4 — What caching changes

时间：5:30–7:30

Without persistence, Spark keeps the transformation plan. When a later action needs the graph data again, Spark can follow the lineage and recompute the missing partitions.

With `MEMORY_ONLY`, Spark stores the computed partitions in memory after the first action. Later iterations can read those partitions directly instead of repeating the same upstream computation.

This gives us two competing effects.

The benefit is less repeated computation in later iterations.

The cost is the first materialization, memory usage, and possible recomputation if a partition is lost.

That is why the experiment must report both the initial materialization time and the subsequent iteration time. If I only report total time, I may hide the reason behind the result.

> [中文解释]

“Materialization”翻译成：“第一次真正执行计算，并把 partition 放进内存。”

“Persistence”翻译成：“把计算结果保留下来，供后续 action 重复读取。”

在这里停一下，让观众看清楚左右两种机制，然后继续。

> [切到第 5 页]

Slide 5 — PageRank workload

时间：7:30–8:45

The workload is PageRank on a directed graph.

For each iteration, every vertex receives rank contributions from its incoming neighbors. The new rank is computed from the previous rank, so the computation repeatedly reuses the graph structure.

The implementation uses a fixed number of iterations and a fixed number of partitions. The graph structure is the part that makes caching meaningful because every iteration reads it again.

I also run a sequential reference implementation on the same input. The distributed Spark result and the reference result should match within floating-point error.

This gives the experiment two evaluation dimensions: performance and correctness.

> [中文解释]

可以补充：“PageRank 不是只跑一次 map 和 reduce，而是重复多轮，所以它适合观察 cache 是否减少重复计算。”

> [切到第 6 页]

Slide 6 — Experiment design

时间：8:45–9:30

I use three generated graph sizes: 2,000 edges, 10,000 edges, and 30,000 edges.

For each size, I run caching disabled and caching enabled. Each run uses six PageRank iterations, four partitions, and two local workers. I repeat each configuration twice and report the mean time.

The experiment records two different timing categories. The first is the initial materialization time. The second is the mean time of the subsequent iterations.

Now I will show one live run, and then I will use the complete repeated experiment results for the comparison.

> [中文解释]

这里强调一句：“现场 demo 只展示机制和一个小样例，最终结论来自三种数据规模、两种 cache 状态和两次重复实验。”

> [准备开始 DEMO。切到终端，不要急着切结果页。]

Live demo — Same workload, cache off and cache on

时间：9:30–12:00

Demo 开始前要说

Now I will run the same PageRank workload twice. The first run disables caching. The second run enables `MEMORY_ONLY` caching.

The input, number of iterations, number of partitions, and local cluster configuration remain the same. Therefore, the cache setting is the main experimental difference.

Demo 操作

在终端输入：

```
cd "C:\Users\xinyuesong\Desktop\CISC3010 presentation\project8-spark"
.\scripts\run_demo.ps1
```

Demo 运行时照读

The script starts a local Spark cluster with one driver and two worker processes.

The first run has `cache=False`. I am looking for the partition count, the number of PageRank iterations, and the runtime of the uncached computation.

The second run has `cache=True`. The important evidence is the reported storage level. It should show that the graph data uses memory persistence.

The two runs should produce the same PageRank output. The cache changes where Spark keeps intermediate data. It does not change the PageRank recurrence itself.

> [中文解释]

这里可以指着终端说：“第一段看 cache=False，第二段看 cache=True；两次的输入和迭代次数相同，区别是第二次保存了 graph RDD。”

Demo 中重点指出什么

当终端显示这些信息时，按下面的顺序指出：

1. `local-cluster[2,1,1024]`：说明本地启动了两个 worker。
2. `partitions=4`：说明输入被切成四个 partition。
3. `iterations=6`：说明这是一个重复使用图结构的迭代任务。
4. `cache=False` 和 `cache=True`：说明只切换了 caching 选项。
5. cached run 的 storage level：说明 Spark 实际保存了 partition，而不是只在代码里写了 cache。
6. `passed: true`、`max_abs_error` 接近 `5e-13`、checksum 接近 `1.0`：说明两次结果都通过正确性检查。

Demo 结束时照读

The live demo shows the mechanism and the correctness check. It does not by itself prove that caching always improves performance. For that, I use the repeated measurements across all three dataset sizes.

> [如果现场运行超过 60 秒，或者出现环境问题，直接说下面这段]

The live run is taking longer than expected, so I will keep the presentation focused and use the saved raw results from the same experiment. The saved results include the cache setting, worker configuration, timing breakdown, and correctness check. I will now move to the comparison.

> [切回 PPT，第 7 页]

Slide 7 — Subsequent iteration results

时间：12:00–13:45

This chart shows the mean time of the subsequent PageRank iterations.

For the 2,000-edge graph, the cache speedup is 0.984 times. This means the cached run is slightly slower in this measurement.

For the 10,000-edge graph, the speedup is 1.005 times. The difference is almost negligible.

For the 30,000-edge graph, the speedup is 1.056 times. The cached run is about 5.6 percent faster for the subsequent iterations.

The pattern is the important result. As the workload becomes larger, the repeated computation becomes more expensive, so the benefit of reusing cached partitions becomes more visible.

Caching helps later iterations, but the improvement is moderate in this local experiment.

> [中文解释]

这里必须讲清楚：“1.056x 是后续 iteration 的 speedup，不是整个程序从启动到结束的总 speedup。”

> [切到第 8 页]

Slide 8 — Materialization cost

时间：13:45–15:15

This slide explains why the small workload can become slower overall.

The cached run pays an upfront cost to compute and materialize the graph partitions. For a small input, the repeated work is already cheap, so the materialization cost can be similar to or larger than the later savings.

For a larger input, the same upfront cost is amortized across more expensive and more repetitive iterations. That is why the larger graph shows a clearer subsequent-iteration improvement.

The result supports a conditional statement: caching is useful when reuse is high enough to pay for materialization and memory overhead.

It would be inaccurate to claim that caching always makes Spark faster.

[中文解释]

“Amortized”可以简单解释为：“一次建立 cache 的成本，被后面很多次重复读取分摊。”

[切到第 9 页]

Slide 9 — Correctness and observability

时间：15:15–16:45

Performance alone is not enough, so I also checked correctness and observability.

The cached and uncached PageRank results both pass the sequential reference check. The maximum absolute error is approximately `4.99 times 10 to the minus 13`, which is floating-point noise for this computation. The rank checksum is approximately 1.0.

The experiment also records the partition count, stage information, cache setting, storage level, initial materialization time, and subsequent iteration time.

These records make the result reproducible. If necessary, the Spark UI can provide a second view of the same behavior by showing stages, tasks, partitions, and storage information.

[中文解释]

可以说：“正确性结果说明 cache 没有改变算法结果；日志和 Spark UI 说明 cache 确实改变了数据保存和复用方式。”

[切到第 10 页]

Slide 10 — Connection to the reading

时间：16:45–17:45

This experiment connects directly to the reading on RDD persistence and lineage.

The reading explains that RDDs can persist partitions so that iterative algorithms reuse computed data. It also explains that Spark keeps lineage information, which allows it to recompute partitions when needed.

My experiment turns those ideas into an observable comparison. The uncached run relies more heavily on recomputation through lineage. The cached run stores partitions in memory and reuses them in later iterations.

The experiment also shows the limitation of the idea. Persistence has a cost, so the best choice depends on workload size, reuse, storage level, and the cost of recomputation.

[中文解释]

这里不用展开文献背景，只要说：“reading 给出机制，实验用 PageRank 和 runtime 把这个机制测出来。”

> [切到第 11 页]

Slide 11 — Closing

时间：17:45–19:15

To conclude, this project asked when Spark caching helps an iterative workload.

I built a reproducible local Spark setup with one driver and two workers. I used PageRank with multiple iterations and compared caching disabled with `MEMORY_ONLY` caching.

The results show three points.

First, caching can reduce the time of later iterations because Spark reuses materialized partitions.

Second, caching has an upfront materialization cost, so it may not help a small workload.

Third, the right conclusion requires both performance measurements and correctness checks. In this experiment, both cached and uncached runs produced the same correct result, while the larger workload showed the clearest subsequent-iteration benefit.

The repository contains the source code, data-generation scripts, experiment scripts, raw JSON results, analysis tables, correctness checks, and the report.

After this presentation, I will add the actual presentation feedback and any requested changes before the final submission.

Thank you. I am happy to take questions.

> [中文解释]

最后可以用中文补一句：“核心结论是，cache 的价值取决于重复使用是否足以抵消第一次 materialization 的成本。” 然后进入 Q&A。

Q&A 备用答案

1. Why was caching slower for the small dataset?

For the small dataset, each uncached recomputation is already inexpensive. The cached run still pays the materialization and memory-management cost, so the upfront cost can be larger than the later savings.

2. Why do the cached and uncached results match?

Caching changes how Spark stores and reuses intermediate data. It does not change the PageRank recurrence or the input graph. Therefore, both runs should converge to the same result, apart from floating-point error.

3. Does this prove Spark is faster on a large production cluster?

No. This is a controlled local-cluster experiment on one machine. It demonstrates the caching mechanism and the tradeoff. A production cluster could add network transfer, skew, failures, and different memory pressure.

4. What happens if a cached partition is lost?

Spark can use the lineage of the RDD to recompute the missing partition. Persistence reduces repeated work when the partition is available, while lineage preserves fault recovery.

5. Why did you use RDDs instead of DataFrames?

This project focuses on RDD persistence and lineage. RDDs make the cache behavior and the iterative graph computation explicit, so they are a direct fit for the project question.

6. How did you make the comparison fair?

I kept the graph generation method, random seed, number of partitions, number of iterations, worker configuration, and correctness check the same. I only changed the cache setting, and I repeated each configuration twice.

7. Why did you separate the first iteration from later iterations?

The first iteration includes cache materialization. Later iterations measure reuse. Combining them would hide the tradeoff between the upfront cost and the repeated-iteration benefit.

8. What are the main limitations?

The experiment uses a local two-worker cluster, a static graph, and a small number of repeated runs. It does not model every production issue, such as executor failure, severe data skew, or cluster-wide network contention.

9. How did AI help with the project?

I used AI assistance for scaffolding, debugging, and organizing the documentation. I ran the code myself, checked the results, verified correctness against a sequential reference, and reviewed the final explanation. The repository includes an AI-use statement.

10. What would you test next?

I would test more iterations, more partitions, larger graphs, and different storage levels. I would also compare local execution with a real multi-node cluster to measure network and memory effects.

一句话提醒

不要在每一页都翻译。只翻译标出的关键词和结论。Demo 的目标是证明“两个 worker、cache 开关、storage level、correctness”，不是现场展示全部代码。